



Transaction Recovery (2)

- commit is an SQL command, and to make sure we can recovery all those committed transaction → we need to have an log record commit (which said that that transaction actually committed)
 - log record that reflect that the commit activity has been performed by user and application, so when you said commit, the system may not has yet return anything → it create commit record in a log buffer, and after a while that commit record output to log file (output successfully → return prompt)

Recap From Previous Class

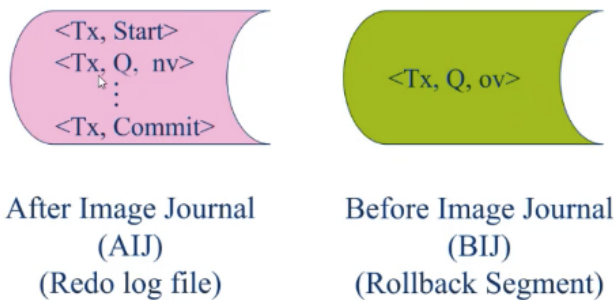
Log-based Recovery

- Problem Statements
 - Problem 1
 - Committed Transactions
 - No Database Modification
 - Problem 2
 - Uncommitted Transactions
 - Database Already Modified
-
- Log based recovery
 - 2 problem
 - committed transaction, not yet output
 - if data still busy in main memory, we normally not move anything to the disk
 - How to recover those committed transaction whose data are loss in main memory.

- uncommitted transaction, output
 - maybe not enough room in memory → output before commit.
 - How to undo those uncommitted modification out of DB Space.

Log file, in principle, should be in stable storage

- We can also separate the old and new value



- Why do we sometimes separate them? (we can keep them together too)



For those failed → system search back log file → can't find the commit record (transaction id placed in undo list) **we use old value to undo DB Space, solve problem number 2**

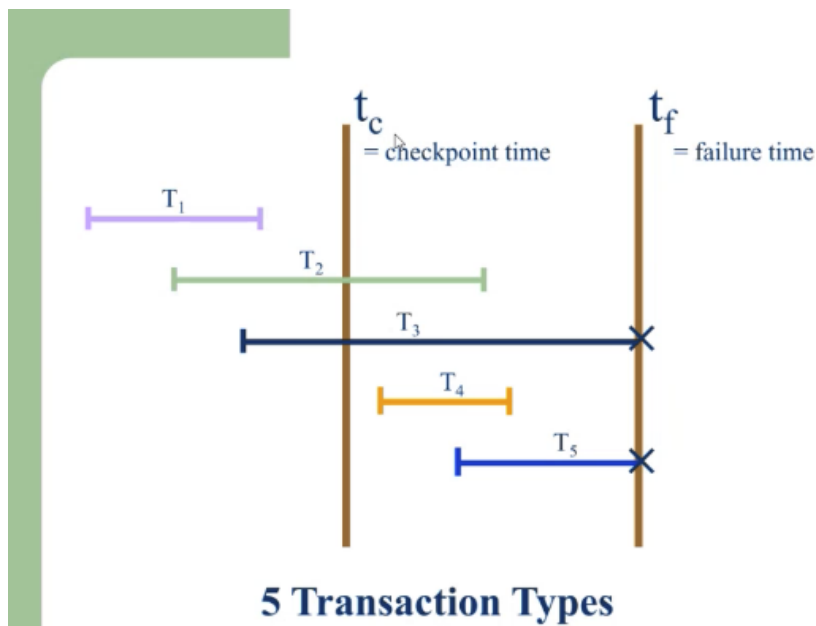


Those has commit record → transaction id place in redo list. **we use new value after each operation in transaction to redo DB space, solve problem number 1**

The point is we don't know which one already output → the point of check point

- check points are required to determine transaction that have already outputs changes to DB space.
-

Content



- at checkpoint, we dump all modified buffer back to the DB space (we can't move directly, we have to output buffer to db space after the log)

Sequence of checkpoint

- if something went wrong, we could recover from log file information.
- in a log file, we have checkpoint log record (it also said checkpoint sequence number, and actual time of a check point)
 - in some system, check point record also show transaction id of those active transaction

Check points

- log record → log file
- DB Buffer → DB Space
- < checkpoint > → log file



at checkpoint time, T2 and T3 still active

- all those modified buffer of T1 is already output → no need to recover

- first part of T2 and T3 (until checkpoint) confirmed to already output to DB space.
- T3 and T5 failed.
- T4 → not confirmed to be output at all
- T3 and T5 failed → start recovery process.
 - use old value of T3 and T5, undo everything (from failure time up to the beginning)
- T2 → redo from the checkpoint time until its commit
- T4 → redo the whole thing



with the checkpoint, recovery time is reduced, we don't have to redo from the beginning, just redo from the checkpoint.

- checkpoint more frequently → shortened the recovery time
- How often checkpoint → you can set
 - if we not set, system has its own default
- Nowadays, system not failed quite often.

at checkpoint, database buffer are forced output to DB space.

before checkpoint, the content in database buffer can output anytime, but we don't know

Some use case

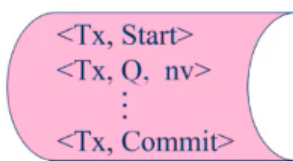


System not failed often. We purchased a new DB Server → staff complain, it failed often → 4 times a year, staff complain already. Another machine, after 2 years never failed. Back to early year, our first machine failed almost every year. (because power system around here not really stable.) (They gave priorities to factory around here) (Quite often there are thunder storm)

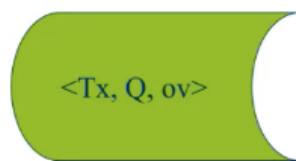
- Why we don't have UPS system → UPS cost more than database server.
- Because of failure, staff ask to buy UPS
- Dean ask why we need an UPS
 - UPS stands for **uninterruptible power supply**. It's a device that permits a computer keep running for a few hours during a blackout or when the primary power source is lost
 - What the main point of having UPS → staff answer him that we need UPS to avoid the damage to the hardware → unstable power could damage the hardware
 - Dean reply back you need a voltage regulator (stabilized power and voltage). → really good voltage regulator too

What is advantage of separating old and new value?

- it can be together, but we can also separate them.
- Most system follows this idea.



After Image Journal
(AIJ)
(Redo log file)



Before Image Journal
(BIJ)
(Rollback Segment)

👉 originally, it was about space. Based on separate log file like this we can recycle old value after each commit. **BIJ size is quite constant size and quite big, big enough to handle transaction**

- ov in BIJ for undo DB space → **nsu** output before commit
 - we need old value because we want to recover uncommitted transaction that failed.
 - after transaction commit, old value no longer used
 - if we separate old value, new value like this → after commit old value can be recycle.
 - (if you combined them, even after commit you still have to keep old value)
- nv in AIJ for redo DB space → **nsu** commit before output
 - we have to keep all those committed record even after commit, because we need new value to redo in case it failed after commit.

We have option to install one of them or both, and still for each option you can still have committed and rolled back logically as usual.

What if the BIJ is full?

- for a transaction that involved lot of data, you may update lot of data, and you need to keep old value in a log file before commit. What if too many that BIJ can't handle.
 - blood group case (20 M → but your BIJ can handle only 1 million)

The transaction that causes the BIJ full problem will be rolled back.

👉 once, there are computer center that try to run that big transaction over night → 40 instructions → the log file is full → system rolled back → example of transaction failures

(Already mentioned)Solution → those 40 instruction are not related, they should not be in the same transaction → set autocommit on.

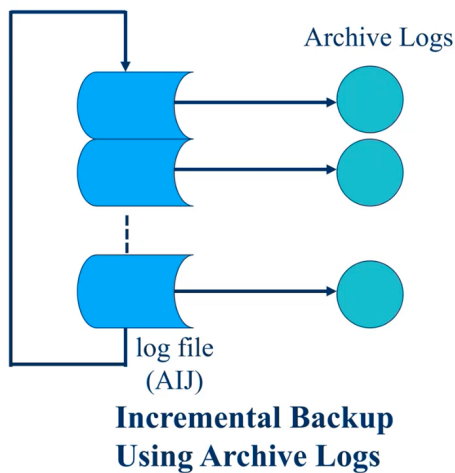
(Solution)

- (not recommended) bypass the BIJ (no log option)(not recommended)
- enlarge the BIJ (online?) (some system allow you to do it online, you can add space to running BIJ real time without shutting down)
- (recommended) separate a big transaction into smaller ones
 - if the instruction is not related. → Fine
 - But if they should be together.
 - ex: blood group update
 - maybe update 1 province at a time, eventually the entire country blood group will be finish.

What if the AIJ is full?

(use multiple AIJ)

- AIJ keeps commit record, so if it full and the system continue working as usual
 - failure occur → you can't recover (no nv for recovery) → you loss the entire system.
 - some system stop working to prevent that
- Some DBMS stop working (MySQL, Microsoft, etc.)
- Some DBMS continue working (Oracle in the past)
- **Expensive system let the machine run, small work group stop working**
- **How to solve this problem (Solution)**



- instead of having only 1 log file, they have multiple AIJ nowadays, so they have 1 active AIJ at a time.
- if the first one is full, the system switch to the next one, and so on.
- You have to estimate how many log file(AIJ) you need in a day.
 - In an unmanned system, no attendance, they set up 8 tape drive, and someone change the tape every morning.
- For back up → we still use tape media **(reusable after certain time)**
(archived media, archived backup)
- the idea is reliable source to keep the data → tape archive (ok)

👉 The solution for when AIJ full is to use multiple AIJ, first one full → next one. And if we don't do anything(don't archive), it will rotate back to the first one and replace the first one.

- one log file rotate back to itself → nooo, at least have 2.
- if you have people attend system all the time → you can have only 1 such archived device, and when the first AIJ is full → you archived it to your archived media. And at that time, the DBMS will use another
- in bank, they use robot to change a tape.

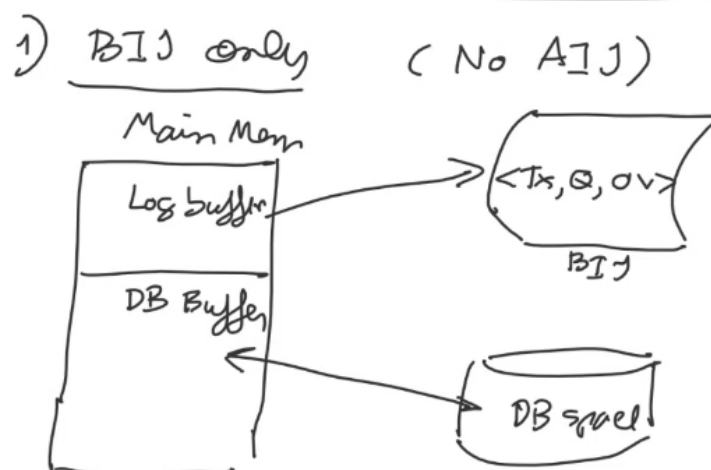
This not only solve the AIJ full problem, it also called incremental backup using archived log. (archived log for AIJ only or for combined)

- Why do we need to archived them?
 - in case of transaction failure or system crash → you can use log record
 - to recover the system incase of disk failure or DB space corruption.
- we have transaction failure, system crash, and disk crash
 - for transaction failure and system crash → we don't need (m?)any backup
 - for media crash → we need backup (the archived log)
 - we need it for disk crash, without the archived log, we at risk.

Log based Recovery Classifications (based on availability)

1. BIJ only (No AIJ)

- one of the most popular → the vendor don't want customer to call them when AIJ is full
- you don't have new value and commit record
- you record only old value without new value



- commit, rollback works as usual logically
- The point is → is How (you don't have AIJ, how does it works)
 - how commit work
 - we don't even have AIJ to put commit record

- without AIJ, transaction commit mean the modified buffer of that transaction must already be output to the DB space completely.
- when you sit in front of your. machine using SQL, update insert delete and then you said commit, the machine return prompt? what happen physically without AIJ? Your modified buffer must be completely output to DB space.
- how roll back work
 - old value from the BIJ can be used

Side effect (AIJ is missing)



Advantage

- No AIJ full problems
- Save some space

- BIJ can also be full, when BIJ full → that particular transaction roll back, not the entire system
- some company throw away machine when log file is full → they don't know about transaction thing



Disadvantage (You don't have AIJ, something must be wrong)

- it about commit (commit now mean → the modified buffer of that transaction must be completely output to the DB space, aj. don't want to use the word output because some system, they don't force output. They just let the output go naturally. → And it could take many hours → **Commit can be very slow — It takes time for the data in the database buffer to be completely output to the DB space**
 - (The DBMS and OS determine when to output (not the application))
 - when you said commit → not commit immediately.
- We don't have AIJ → no checkpoint

- consequence of slow commit, during transaction before commit(end of transaction), your transaction may use many data and the system may lock some table/data. So if your transaction have slow commit, people wait more. Wait for you to commit because before commit, you lock resources. If you commit quickly people can use resource more quickly and throughput is better.



Use case: a vendor install a system free of charge to a potential customer, the customer wrote the small program

```

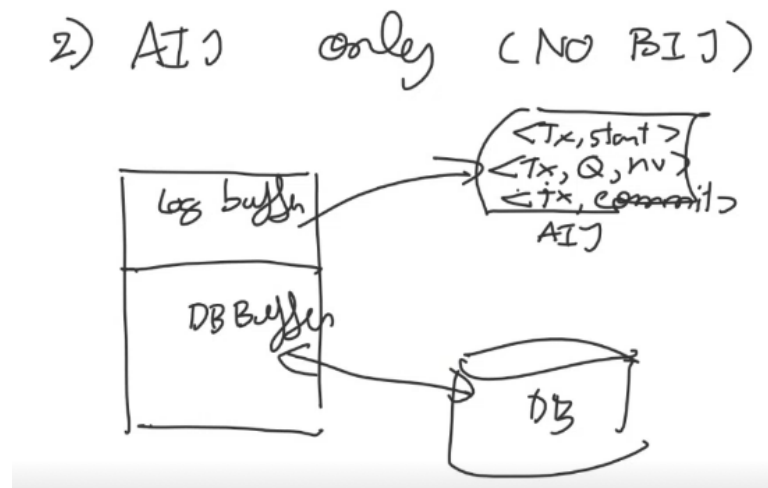
begin tx
30,000 times [ Insert into T1 values( )
               display (date,time) ]
end tx (commit)
display (date,time)

```

- the commit time between last insert and the commit took them 18-19 hours (test system).
- insert 30,000 rows took 18 hours to commit.
- they try it on biggest machine from that vendor took 5 hours.
- Problem → if you have to long commit time → your system doesn't have an AIJ.
 - with an AIJ, the commit record go to AIJ → finish
 - without AIJ, depend on DBMS and OS.
- Solve → Try to create AIJ and check if it work → when added 19 hours went down to 8 minutes.
 - at first they refused → they said with log file, you have to output to both log and disk → double or more I/O. so they said it will be slower → not true.

- in reality, to output log record it just like add new block to the end, but to move data from DB buffer to DB space require careful placement. you have to put in correct way so you can retrieve them easily later, require correct placement and index. So it take more time that output log buffer to log file.
- with log file AIJ, commit a lot faster.

2. AIJ only (No BIJ)



- commit and rollback work as usual logically
- How it work?
 - commit → (as usual) transaction commit go to log file
 - roll back(no old value how does rollback work) you don't have any old value to undo. How to solve second problem (output before commit)
 - we don't know when it was last update, maybe 10 years ago?
 - (Prevention) do not allow output before commit
 - this kind of DBMS won't allow output before commit
 - Old value is not required.
 - During the transaction operation, only log record go to log file → after commit, we start the transfer
- have checkpoint (special kind) → if you checkpoint → it will output only committed one.



Advantage:

- Fast commit (only deal with AIJ, won't touch the DB space)
 - sometime, this call fast commit server
 - popular among workgroup servers.



Disadvantage (you don't have BIJ, you can't expect everything to be normal)

- Not good for large amount of data
 - The DB buffer could be full before a transaction commit → system crash.
 - Do not allow output before commit, what if you have large amount of data → what if your data is too big for your database buffer and you can't output anything. The buffer will be full before commit → system stop working

- If you want fast commit and have to deal with large amount of data, you need both AIJ and BIJ

3. Both AIJ and BIJ

- fast commit (use AIJ)
- can deal with large amount of data (use BIJ)